Nexus – Shared Multiplayer Agentic Workspace

Summary

Nexus is a multiplayer AI workspace: a team and specialist AI agents work in one live conversation across GitHub, Slack, Linear, Notion and Figma. Teams build their own agents, workflows and shared artifacts on top. Its core rule is enforced in the server: **agents can propose, but only a permitted person can make anything happen.**

- **Stage:** working product deployed on Oracle Cloud; pre-revenue
- **Built:** 5 integrations, 5 specialist agents plus a coordinator, custom agents, workflows, artifacts, approvals and an audit log
- **Quality:** 44 test files with about 376 test cases, run in CI against a real Postgres

The problem

Work is multiplayer, but most AI is single-player: private chats, context pasted in by hand, and no record the team can see. Giving agents real access to team tools raises four questions a security team must answer first. Who allowed this action? Whose access did it use? Can untrusted text steer it? Can we reconstruct what happened? Without answers, companies either block agents or let them in without controls.

The product

- **Shared chat:** one live conversation with presence, @-mentions and handoffs.
- **Specialist agents:** GitHub, Slack, Linear, Notion and Figma agents, each limited to its own tools, plus a coordinator.
- **Build your own:** custom agents, workflows (manual, scheduled or event-triggered) and artifacts such as release notes.
- **Governance:** approval cards for every write, Admin and Editor roles, shared memory with freshness dates, an audit log, and run tracing with cost.

Why now

- Gartner forecasts that 40% of enterprise apps will include task-specific AI agents by 2026, up from under 5% in 2025.
- In NeuralTrust's June 2026 survey of 160+ CISOs, 72% of organizations were implementing or scaling agents, but only 10% had them in full production. 73% of CISOs were very or critically concerned about agent risk.
- Of reported agent incidents, 68% involved prompt injection, 61% leaked sensitive data and 52% involved unauthorized actions.

Nexus targets that gap: teams that want shared agents but need controls that pass security review.

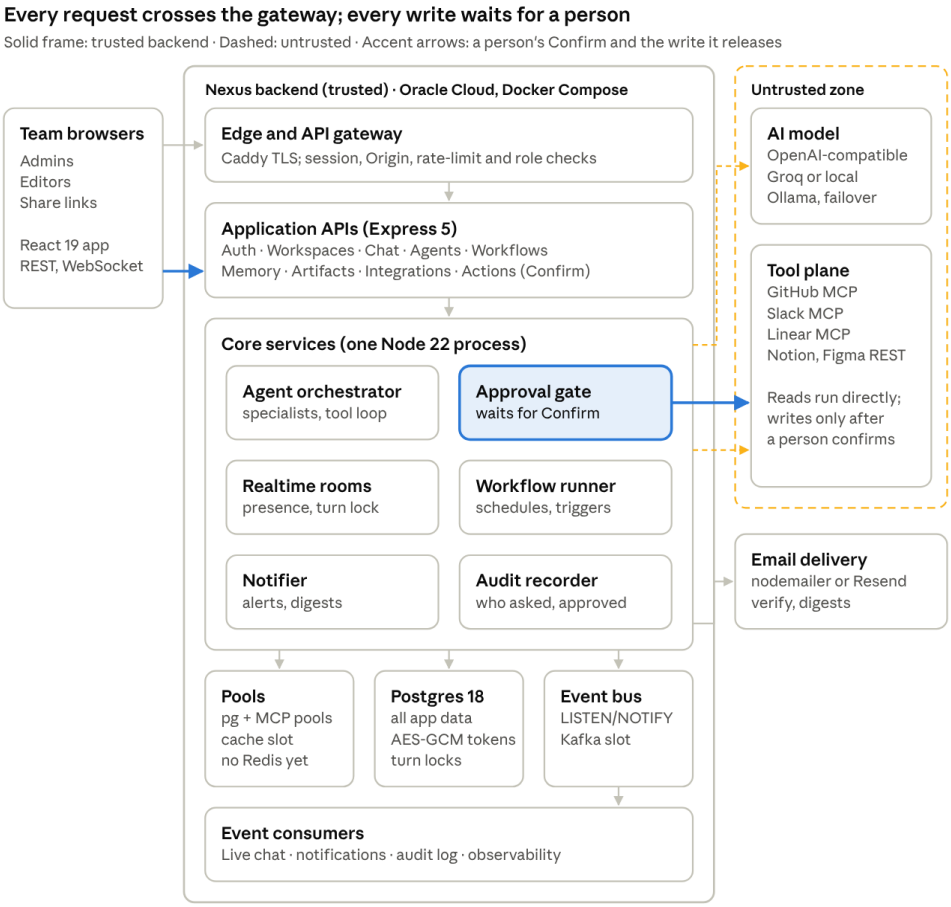
Competition

Player	Strength	Nexus's angle
Dust	$40M Series B, 3,000+ customers, 70+ connectors	A hard approval gate on every write; agents use only the requester's credentials
Slack and GitHub Copilot in Slack	Free to existing customers	Neutral across five tools, under one approval and audit layer

Player	Strength	Nexus's angle
Microsoft 365 Copilot	Distribution	Not tied to one vendor's suite
YC QM (open source)	Free and self-hosted	A finished product with roles, approvals and audit

Shared chat isn't the moat; the governance layer is.

Architecture at a glance



Nexus architecture · trusted backend layers, untrusted model and tools

Requests pass the gateway checks and the APIs. The orchestrator works with the AI model and reads from tools, and any write waits at the approval gate for a person's Confirm. Postgres holds all data and also carries live events between servers.

Security and threat model

Everything the AI model writes and everything a tool returns is treated as untrusted, so the controls live in Nexus's server, not in the prompt.

Threat	Control	Remaining risk	Status
Injection triggers a write	Human approval of every write	A harmful proposal gets approved	Partly mitigated

Threat	Control	Remaining risk	Status
Injection leaks data	One provider's tools per agent; previews	Leaks within one provider	Partly mitigated
Borrowed access	Requester's own credentials only	None known	Mitigated
Cross-workspace access	Workspace-scoped queries and checks	None known	Mitigated
Privilege escalation	Server-enforced roles	None known	Mitigated
Compromised tool server	Unknown tools treated as writes	Image not pinned	Open
Credential theft	AES-256-GCM, hashed sessions	One key, no rotation	Partly mitigated
Runaway agent	Turn caps and token guard	No spending cap	Partly mitigated

Prompt injection

Nexus contains injection rather than detecting it. Injected text can shape what an agent says or proposes, but it can't trigger a write, approve itself, reach another provider's tools, or act in a read-only workflow. Next: risk-scored approval cards, screening of retrieved text, and a red-team test suite.

Information leakage

Workspaces are isolated, and that is covered by tests. The main open risk is inside a conversation: an answer built from one member's private access is visible to everyone in it, so private replies are planned. Hosted models receive conversation context; teams can run a local model instead, and secret and PII redaction is planned. Public artifact links are opt-in and revocable.

Access leakage

Agents have no access of their own. Each turn borrows the requester's OAuth connection, checked again at confirm, under Admin-set role and provider permissions. Workflows re-check their owner on every run. Next: scoped grants (for example, "branches only, never main"), per-resource policies, and SSO with SCIM.

Test coverage

44 test files with about 376 test cases run in CI against a real Postgres. Only the AI model and provider calls are mocked, and 25 tests are labeled as security tests.

- **Tested:** workspace isolation, role enforcement, permission requests, the approval lifecycle and audit, failing closed at confirm, read-only workflows, credential handling, and origin and session checks.
- **Not yet tested:** a dedicated test that agents use only the requester's connection, refusal of a second confirm, chat text never approving, prompt injection payloads, and load. Line coverage isn't measured yet.

Roadmap

Security hardening comes before enterprise and scale

Proposed sequence; timing to be set with the funding plan

